

시키지도 않은 시공비 청구서

주장은 유지 . 근거는 정확하게 .

영어 활용 · KISS/DRY/YAGNI · 설명 최소화 · 결정된 스펙 주입

불필요한 출력 · 사고 · 재전송을 줄이면 비용을 낮출 수 있다 .

이 자료는 실측 , 로그 재계산 , 비용 시나리오를 구분한다 .

숫자마다 증거의 종류를 붙인다

측정 ≠ 가정 ≠ 인과 효과

API 저장 로그 : 80 건 언어 비교 + 30 건 사고 예산 스왑

새 로컬 실측 : 대응 번역문 1,012 쌍의 토큰나이저 인코딩

브라우저 · mock : 코드 동작 검증 / 비용 시나리오 : 정책 효과 실측 아님

이번 정정의 새 유료 API 호출은 0 회다 .

요금은 모델 ID 와 단가를 함께 본다

\$0.25 / \$1.50

Gemini 3.1 Flash-Lite · Standard text · USD / 1M tokens

출력에는 `thoughtsTokenCount` + `candidatesTokenCount` 를 포함한다 .

기존 \$1.25/\$10 은 발표 환산 가정으로 보존한다 . Flash-Lite 공식 과거 가격이라는 뜻은 아니다 .

$\text{usage} \times \text{리스트 단가}$ 는 청구서가 아니다 . 무료 티어 · 계약 · 세금 · 도구료는 별도 .

대응 번역문에서는 한국어 토큰이 더 많다

1.4723× / 2.3669×

o200k_base: EN 26,873 → KO 39,566

cl100k_base: EN 27,182 → KO 64,338

FLORES-200 devtest 의 대응 번역 1,012 쌍 전부를 문장별 인코딩 , 토큰 합계의 비율 .

동일 단가에서의 토큰 비용 이점을 지지한다 . Gemini·Claude 배수로 옮기지 않는다 .

짧은 한국어가 적은 의미를 뜻하지 않는다

한국어 글자 수 = 영어의 49.99%

대응 번역문인데 한국어 문자는 약 절반이다 .

\$/1,000 자는 문자 단위 지표이며 언어 간 의미량을 맞춘 지표가 아니다 .

‘한국어가 일을 덜 했다’는 결론은 글자 수만으로 낼 수 없다 .

호출당 비용과 문자당 비용은 다른 질문

둘 다 보고, 한쪽을 착시라 부르지 않는다

reason / Standard: KO/EN 호출당 0.729×

reason / 기존 환산 : 호출당 0.728×, 문자당 1.36×

같은 입력 과제라도 생성 내용 · 답의 품질 · 추론 언어까지 같다고 확인하지 않았다 .

실험은 과제 2 개 × 언어 2 개 × 20 회 반복이다 . 일반 언어 벤치마크가 아니다 .

영어 추론의 이점은 정확도로 뒷받침한다

83.7% vs 75.4%

Language Matters (2025), 표 1: 한국어 MATH-500 입력, 영어 / 한국어 prefilling

평가한 네 모델 평균. 모델별 차이가 크며 모든 모델 · 도메인의 우위는 아니다.

저장소의 사고 토큰 수가 많다는 사실만으로 ‘더 깊거나 정확하다’고 증명하지 않는다.

영어 활용 권장은 유지하되 자기 과제의 정확도 · 비용으로 확인한다.

간단한 계산에도 사고가 비용을 크게 늘린다

약 199.47 배 / 기존 240.02 배

자동 (-1) / 미지정 비교 . 같은 문제를 조건당 6 번 실행한 저장 로그 .

$\pm \$0.05$ 허용오차 통과는 각각 6/6. 일반 정확도의 동등성 증명은 아니다 .

자동 / 명시적 0 은 Standard 195.97 배 . 명시적 0 은 5/6.

240 배 메시지는 기존 환산 단가라는 조건을 붙여 유지한다 .

미지정과 0 을 합치지 않는다

6/6 · 5/6 · 2/6 · 6/6 · 6/6

순서 : 미지정 / 명시적 0 / 512 / 2048 / 자동 (-1), $\pm \$0.05$ 기준

정확히 \$38.016 기준은 각각 6/6 · 5/6 · 1/6 · 4/6 · 3/6.

6/6 의 Wilson 95% 구간도 약 61-100% 다 . 동등성 검정이 아니다 .

512 조건은 모두 STOP, 사고 127-154 토큰 . ‘예산에 잘려 오답’이라는 원인은 미확인 .

Welch 검정은 t 분포를 쓴다

$$t = 6.188 \cdot \text{자유도} = 35.713$$

reason / 문자당 / 기존 단가: 정규근사 $6.08e-10 \rightarrow$ Welch $4.02e-07$

평균 차이의 표준오차를 표본에서 추정하므로 정규분포보다 꼬리가 두꺼운 t 분포를 사용 .

$p = 2 \times t.sf(|t|, df)$. 1-CDF 는 작은 꼬리에서 정밀도를 잃을 수 있다 .

p 값은 수정해야 한다 . 유의성의 소멸을 뜻하지는 않는다 .

교정 후에도 비용 차이의 유의성은 유지

네 비교 모두 Bonferroni(4) 통과

explain 호출당 : $p = 1.60e-03$

explain 문자당 : $p = 3.00e-10$

reason 호출당 : $p = 7.80e-04$

reason 문자당 : $p = 4.02e-07$ — 모두 기존 \$1.25/\$10 단가 . 반복 호출의 조건부 검정 .

네 가지 원칙은 그대로 쓴다

필요한 결과만 , 적합한 설정으로

- ① 영어는 토큰 효율과 추론 정확도에 유리할 수 있다 . 사용자에게는 필요한 언어로 답한다 .
- ② KISS/DRY/YAGNI: 불필요한 코드 · 전체 재출력 · 재작업을 줄인다 .
- ③ 설명 길이와 사고 예산을 과제에 맞춘다 . 정확도 게이트를 함께 둔다 .
- ④ 스펙에 결정을 먼저 적는다 . 스펙 작성 비용과 실제 재작업 변화를 함께 잔다 .

DRY: 전체 파일 대신 변경분을 받는다

5,000 → 300 토큰

10 회 수정에서 출력만 50,000 → 3,000 으로 줄인다는 가정이면 94% 절감.

실제 패치가 적용되고 테스트를 통과해야 절감이 성립한다.

생성 토큰 감소를 개발 시간 · 품질 동등성의 실측으로 바꾸어 말하지 않는다.

압축과 캐싱은 대체재만이 아니다

\$0.4014 → \$0.2007

Sonnet 단가, 프리픽스 12,000, 100 회: 첫 쓰기 + 이후 99 회 캐시 읽기.

원문 캐시 \$0.4014 / 절반 압축 + 캐시 \$0.2007.

압축 비용 · 품질 · 최소 길이 · TTL 조건을 만족해야 한다. 요약 생성 비용은 이 표 밖이다.

출력 대신 결정된 입력에 비용을 쓴다

단가 차이는 지렛대, 품질은 별도

출력 단가가 더 높으면, 짧은 출력 대신 조금 긴 입력이 유리할 수 있다.

입력이 늘어난 만큼 출력을 실제로 줄였는지 usage 로 확인한다.

‘입력 3 배 · 비용 66% 절감’은 exp02 의 특정 토큰 가정이지 보편 실측률이 아니다.

캐시 적격성은 모델별로 다르다

Haiku 4.5: 최소 4,096 토큰

Sonnet 4.6: 1,024 / Opus 4.6: 4,096 (공식 문서 확인일 2026-09-20)

짧은 프리픽스를 1K 공통 규칙으로 할인하지 않는다 . 미확인 기준은 ‘미확인’으로 둔다 .

적격성 충족은 히트 보장이 아니다 . 실제 cache usage·TTL·쓰기 / 저장료를 본다 .

캐싱의 손익은 조건으로 재현한다

기본 시나리오 62.35% 절감

고정 20,000 + 가변 1,000, 출력 1,500, 100 회, Sonnet: \$8.550 → \$3.219

히트 0, 매번 쓰기 가정은 \$10.050. 캐시 없음보다 비쌀 수 있다.

‘캐싱 90%’는 캐시 읽기 단가 할인이다. 전체 세션 절감률과 다르다.

20.8 배는 페르소나 시나리오로 보존

기존 20.8 배 / 수정 기본 19.0 배

기존 : A 6 턴 / B 2 턴 , 출력 확대 2.9 배 , effort 0.35, 한국어 사고 승수 1.44 등 가정 .

기본 : 사고 언어 승수는 1.0, 최소 길이 미달 캐싱은 미적용 .

두 조건 모두 일부 실패일 인코딩이 포함된 비용 모형 . 실제 품질 동등 A/B 가 아니다 .

`python demo/compare_personas.py --scenario legacy` 로 기존 발표 수치 재현 .

SDD 절감은 '턴 감소' 가정에 달린다

기존 71.0% / cold-cache 68.7%

40 턴 → 12 턴은 모델의 입력 가정 . 스펙이 실제 이만큼 줄었다는 관측은 없다 .

스펙 작성 : 입력 3,000 + 출력 4,000 비용을 이미 포함한다 .

기존 약 71% 는 캐시 예열 비용 제외 . 기본은 사이클별 첫 쓰기까지 포함한다 .

주장 유지 : 짧고 결정적인 스펙을 우선 시도하되 재작업 · 품질 · 총비용으로 검증한다 .

카페 케이스 : 절감 관측의 증거 수준

usage 추정치 환산 -8.3%

고정 usage 추정치 + GPT-5 단가 : \$0.07135 → \$0.06540

토큰 합계 7,868 → 8,010 (+1.8%). 실제 청구 usage 가 아니라 세션 추정치다 .

캐시 적격성 미확인 . 모든 추정 입력을 신규 입력으로 환산하면 -8.1% .

Spec 은 Vibe 결과를 역산 . 백지 스펙 비용 · 재작업 감소 · 품질 동등성을 검증한 것은 아니다 .

반례의 비교 대상을 정확히 읽는다

SDD 도구 간 비교 $\neq$ SDD 대 무스펙

Spec-Kit +97~109%: OpenSpec 대비 두 사례의 외부 블로그 보고 . 둘 다 SDD 접근 .

ETH AGENTS.md 연구 : 저장소 컨텍스트 파일 조건 . 모든 짧은 스펙이 손해라는 실험 아님 .

짧고 필요한 요구사항만 쓰라는 실무 원칙은 유지하되 효과를 직접 측정한다 .

절감률을 쌓을 때도 가정을 드러낸다

85.825%

잔여비용에 30%, 40%, 50%, 25%, 10% 를 순차 적용한 산술 결과 .

85.825% 는 60-80% 범위 밖이다 . ‘일치한다’는 표현은 삭제한다 .

레버의 중복 · 상호작용 · 추가 비용을 검증하지 않았다 . 경험적 상한도 아니다 .

서브에이전트는 전체 비용으로 비교한다

메인 + 자식 + 요약 비용

동일 톨 작업 가정 : naive \$5.6010, compact \$3.4425, subagent \$2.7360

20 턴 , 매 턴 톨 8,000, 초기 5,000, 출력 300, 요약 1,500, 같은 Sonnet 단가 .

기존 -87% 는 자식 · 요약 비용이 빠진 메인 컨텍스트 계산이었다 .

동일 작업량 가정이어도 요약 품질 · 실패 · 재시도까지 같다고 검증한 것은 아니다 .

라우팅은 단어보다 작업 목적을 본다

파일 업로드 '구현' → IMPLEMENT

설계 · 구현 · 설명 목적을 파일 · 검색 · 변환 같은 부수 단어보다 먼저 본다 .

구성한 반례 테스트는 회귀 검증이지 실사용 오류율 추정이 아니다 .

기본 tier 단가는 모델의 공식 가격이 아니라 명시적 비용 시나리오다 .

사다리 재생은 정책 A/B 실측이 아니다

리플레이는 유용하다 . 명칭을 바꾼다 .

기존 3.06 배는 고정 발화를 다른 정책으로 처리한다는 비용 재생이었다 .

지금은 respec 의 무료 중단과 실행 비용을 분리하고 , 총 입력 증가분을 한 번만 더한다 .

A· 오케스트레이터는 범위 · 리셋 · 출력 상한 · 진단 후 적용 비용을 공통 함수로 계산 .

새 task_id 의 의도 이력을 분리해 앞 작업의 재작업 상태가 새 작업으로 새지 않게 한다 .

검증기가 실패를 놓치지 않게 한다

거짓 통과를 반례로 검사

슬라이드 240 → 999 변조를 거부 : 실제 PPTX 의 모든 텍스트와 생성 근거를 대조 .

모바일 1 열 → 2 열 변조 , 동작 없는 메모 HTML 을 브라우저 AC 가 거부 .

실패 assertion 은 exit 1. shell pipefail 로 파이프 뒤에서도 실패를 보존 .

mock 은 요청 구조 · 계산 검증일 뿐 새 live 측정으로 소개하지 않는다 .

API usage 를 다룰 때의 세 가지 규칙

캐시를 두 번 빼지 않는다

Gemini promptTokenCount 는 캐시 입력을 포함 . cost() 에 전체 prompt 와 cached 를 전달 .

Gemini Content 는 role + parts. 모델 응답의 thoughtSignature 와 functionCall id 를 보존 .

툴 내용은 두 번째 요청에서 처음 입력되고 세 번째 요청에서 재전송된다 .

요청 실패 · 절단 · 키 없음은 미검증 / 실패로 기록 . 3 번더 ‘확인됨’을 미리 출력하지 않는다 .

숫자를 직접 확인할 수 있게 남긴다

원자료 · 산식 · 기준일 · 재현 명령

results/audit_metrics.json: 저장 로그 통계 + 로컬 측정 + 시나리오를 분리한 지표 .

docs/CORRECTIONS.md: 무엇을 유지하고 무엇을 바꿨는지 상세 설명 .

SOURCES.md: 공식 단가 , 논문 표 , 외부 케이스의 비교 조건 .

원본 110 행은 보존 . 없는 원자료를 재측정 없이 만들어 넣지 않는다 .

작게 바꾸고 , 자기 usage 와 품질로 확인한다

영어 · 간결함 · 스펙 · 컨텍스트 관리

주장은 유지한다 : 불필요한 생성 · 사고 · 재전송은 줄일 가치가 있다 .

배수는 모델 · 단가 · 과제 · 시나리오에 붙여 말한다 . 절감만으로 품질을 대신하지 않는다 .

정확도 · 재작업 · 총비용을 함께 측정해 실제 배포 정책을 선택한다 .